



Tutorial S1G: Hash Table (Solutions)

Question 1

1) Hash(Key) = Key mod 11

The Hash function can return a value of 0, but the hash table is index from 1 to 11.
The function should be changed to Key mod 11 + 1.

2) WHILE Table[Index] <> Key

Potentially result in infinite loop, if key not present in hash table and no empty slots.

Amend code to:

Count ← 0

WHILE Table[Index]<>Key AND Table[Index]<>NULL AND Count < 11

Index ← Index + 1

Count ← Count + 1

3) When index increases more than the table size, it should wrap over and restart from 1. Amend code to: **Index ← (Index + 1) mod 11**





Question 2

- (a) Find index using hash function.

Check if index is already occupied. If not occupied, insert key there and done.

If occupied, find next available spot using quadratic sequence: 1, 4, 9, 16.

Keep finding next available slot until no slot left. Then maybe can resize hash table.

- (b) Linear probing use sequence: 1, 2, 3, 4, etc, while quadratic: 1, 4, 9, 16, etc.

This helps to spread out probing and reduce clustering.

For example, for a hash table size of 37, and calculated index is 3, then linear probing will be 4, 5, 6, 7, etc, while quadratic probing will be 4, 7, 12, 19, etc

- (c) Invalid collision resolution method.

Consider an example of a hash table with 3 elements:

1	NULL	3
[1]	[2]	[3]

Insert data = 1, $\text{hashValue} = 1 \% 3 = 1$

Iteration 1: $\text{hashTable}[1] \neq \text{NULL}$, so $\text{hashValue} \leftarrow 1 + (-1)$,
since $\text{hashValue} < 1$, **$\text{hashValue} \leftarrow 3$** , and $\text{step} \leftarrow (-1) * (-2)$

Iteration 2: $\text{hashTable}[3] \neq \text{NULL}$, so $\text{hashValue} \leftarrow 3 + 2$,
since $\text{hashValue} > 3$, **$\text{hashValue} \leftarrow 1$** , and $\text{step} \leftarrow 2 * (-2)$

Iteration 3: $\text{hashTable}[1] \neq \text{NULL}$, so $\text{hashValue} \leftarrow 1 + (-4)$,
since $\text{hashValue} < 1$, **$\text{hashValue} \leftarrow 3$** , and $\text{step} \leftarrow (-4) * (-2)$

Iteration 4: $\text{hashTable}[1] \neq \text{NULL}$, so $\text{hashValue} \leftarrow 3 + 8$,
since $\text{hashValue} > 3$, **$\text{hashValue} \leftarrow 1$** , and $\text{step} \leftarrow 8 * (-2)$

The while loop will continue and enter an infinite loop, as hashValue alternates between 1 and 3. Thus collision resolution is invalid.